

Astro + Cloudflare R2: Auto-Rebuild Architecture

Generated with GitHub Copilot

2026-10-01

Astro + Cloudflare R2: Automated Rebuild Architecture

Goal

Store all media files, JSON data, SVGs, and videos in a Cloudflare R2 bucket. When content is updated through the Cloudflare/R2 API, automatically trigger an Astro static site rebuild so the published site reflects the new content.

Verdict: Yes, this is possible and is a standard JAMstack “rebuild on content change” pattern.

Architecture Diagram



1. Storage Layer: Cloudflare R2

- Create one or more R2 buckets, e.g. `site-assets`, organized by prefix:
 - `media/`, `data/`, `videos/`, `icons/`
- Manage objects via:
 - The **S3-compatible API** (works with AWS SDK v3, `boto3`, `aws-cli --endpoint-url`), or
 - The **native R2 API / Wrangler CLI** (`wrangler r2 object put/get/delete`)
- Expose content for the Astro build in one of two ways:

- **Public bucket / custom domain:** simplest, lets Astro `fetch()` assets over HTTP at build time.
- **Private bucket + R2 binding:** more secure, accessed only from a Worker or from the build process using S3 SDK credentials stored as secrets.

2. Change Detection: Triggering a Rebuild

R2 has no direct “notify Cloudflare Pages” feature, so bridge it with one of:

Option A — Event Notifications + Queue + Worker (recommended for decoupled updates)

1. Enable **R2 Event Notifications** on the bucket for `PutObject` / `DeleteObject`.
2. Notifications are delivered to a **Cloudflare Queue**.
3. A **Worker** consumes queue messages and calls the Pages **Deploy Hook URL**.
4. This works even if updates come from third parties or multiple scripts, since the trigger is bucket-driven, not caller-driven.

Option B — Direct call from your update script (simpler)

1. Your script/API call that uploads/deletes objects in R2 also performs a second `fetch()` call to the **Deploy Hook URL** right after the R2 operation succeeds.
2. No Queue/Worker needed, but every caller must remember to trigger the hook.

3. Build Layer: Astro Reading from R2

During `astro build`:

- **JSON data:** fetch via `getStaticPaths()` or `frontmatter`, e.g. `const res = await fetch('https://assets.example.com/')`
- **Images / SVG / video:**
 - Reference the public R2 URL directly in `<img>` / `<video>` tags (no bundling needed), or
 - Download assets into `src/assets/` or `public/` via a pre-build Node script if you want Astro’s built-in image optimization (`astro:assets`) to process them.
- **Private bucket access:** use AWS SDK v3 (`@aws-sdk/client-s3`) configured with the R2 endpoint and access keys, stored as **Cloudflare Pages environment variables / secrets** — never committed to the repo.

4. Hosting & Deployment: Cloudflare Pages

- Connect the Astro repository to **Cloudflare Pages**.
- Pages runs `astro build` on every trigger and deploys the static output to Cloudflare’s global CDN.
- Generate a **Deploy Hook** under *Project Settings* → *Builds & deployments* → *Deploy Hooks* (per-branch secret URL). Triggering it:


```
curl -X POST "https://api.cloudflare.com/client/v4/pages/webhooks/deploy_hooks/<hook-id>"
```

5. Security Considerations

- Keep R2 write credentials server-side only (Worker secrets or CI/CD secrets); never expose to the browser/client.
- If the bucket must be public for build-time fetches, confirm no sensitive data is stored there.
- Treat the Deploy Hook URL as a credential — anyone with it can trigger a rebuild (low risk, but avoid leaking it publicly).
- Validate uploaded file types/sizes in the update script before writing to R2, to avoid unexpected content reaching the build.
- Use least-privilege R2 API tokens (scoped to the specific bucket, read/write only as needed).

6. Summary Flow

1. R2 bucket holds all assets; updated via S3/R2 API.
2. R2 Event Notification → Queue → Worker (or direct script call) triggers the Cloudflare Pages Deploy Hook.
3. Cloudflare Pages reruns `astro build`, fetching current R2 content.
4. New static site is deployed automatically to Cloudflare's CDN.